

node.jsを効率的に管理する環境を構築する

Homebrewのインストール

mac自体のパッケージが煩雑にならないように、コマンドプロンプトで本体で動作するパッケージマネージャーをインストールする。

アプリケーション > ターミナル。

本体のマネージャーのためnode専用ではなく、既に導入済みの場合はスキップして良い。
パスワードを聞かれた場合は入力する。

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

コンソールに表示される内容を読んで次の内容を決定する。

ほぼデフォルトアプリ以外入っていない当方のiOS26では以下の内容になっており、コマンドパスを通すように提案されている(パスは端末名を参照)。

Run these commands in your terminal to add Homebrew to your PATH:

```
echo >> /Users/mms-apple-pc/.zprofile
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> /Users/mms-apple-pc/.zprofile eval
"$(/opt/homebrew/bin/brew shellenv)"
- Run brew help to get started
- Further documentation:
https://docs.brew.sh
```

```
# コンソールの内容に従い、以下のパスを置き換えて実行すること
echo >> /Users/mms-apple-pc/.zprofile
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> /Users/mms-apple-pc/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

Homebrewが入っているかどうか確認する。

バージョン情報を確認するコマンドを実行することで、余計な処理が実行されず安全に確認できる。

```
brew -v
```

Voltaのインストール

コマンドプロンプトで以下を実行し、Homebrew経由でVoltaをインストールする。

<https://formulae.brew.sh/formula/volta>

```
brew install volta
# listコマンドを使うことで、brew経由でインストールしたアプリが確認できる
brew list
```

node.jsのインストール

Voltaを経由してnode.jsをインストールする。
長期の安定サポートであるLTSをメインとして設定する。

```
# LTS版の場合はバージョンの指定は不要
volta install node
```

Voltaのコマンドを細かく知りたい場合は以下のページやmanコマンドで、同ドメインサイトにて解説が記載されている。

<https://docs.volta.sh/reference/>

コマンドプロンプトからnode.jsやnpmを実行できるようにしよう

Voltaをインストールしただけでは、Volta経由でインストールしたパッケージのコマンドを実行ができない(npm ○○といった形式でコマンドを実行したい)。
グローバルにnodeを入れる方法も簡単ではあるが、せっかく本体の環境を汚さずに入れたので、Voltaに対してパスを通すようにシェルの方を設定する。

```
# 以下は実行できない状態になっている
node --version
npm --version
```

```
# 上書きと追記コマンドは似ているのでコマンドはコピペで対応しよう
echo 'export VOLTA_HOME="$HOME/.volta"' >> ~/.zshrc
echo 'export PATH="$VOLTA_HOME/bin:$PATH"' >> ~/.zshrc
```

また、以下のコマンドを実行し、作業中のシェル(zsh)にパスの設定を反映する。
再起動でも反映されるが、確実に反映できるように実行しておいた方が無難。

```
source ~/.zshrc
```

Voltaパスを通したことにより、以下のコマンドが実行できるようになる。

```
node --version
```

```
npm --version
```

node.jsのプロジェクトを動かしてみよう

任意のディレクトリに移動し、nodeのプロジェクトを作成する。

実行場所に依存せず立ち上げが可能なため、JSのプロジェクトだとわかりやすい場所に設定しよう。

VSCoide上のターミナルでも同じように動かせるので、ディレクトリを決めたらVSCoideでその場所を開く。

プロジェクトの初期設定をしよう

任意の場所に作成したディレクトリ(直下)をVSCoideで開き、ターミナルを起動する。

起動したターミナルで以下のコマンドを実行する。

```
npm init
# プロジェクトの詳細について質問されるので、適宜回答を記載する。
# JSONで設定されるファイルで後でも変更でき、またサンプルでもあるので今回は深く気にする必要はなし。
# 以下サンプル用として。
# "name": node-sample-first
# "version": {空エンター}
# "description": nodeの初期サンプルプロジェクト
# "main": {空エンター}
# "scripts": {空エンター}
# "author": {空エンター}
# "license": {空エンター}
# "type": {空エンター}
```

Voltaでバージョンのピン留めをしてみよう

作成したpackage.jsonを確認しながら、Voltaでピン留めした場合の動作を確認してみよう。

```
# 実行時点で変わる可能性あり
volta pin node@lts
volta pin npm@11.6.1
```

package.jsonにVoltaの項目が加わり、これを共有することで、他の人と開発環境を揃えやすくなる。

サンプルプログラムを動かしてみよう

サンプルに使うパッケージをインストールする。

インストールしたパッケージはJSONに記録され、「node_modules」に格納される。

node_modulesは大容量かつ大量のファイルになるため、gitではJSONのみを共有し、各環境で「npm install」を実行することで復元する。

- sass
 - cssを効率的に記述するツール
- express

- サーバーサイドでJavaScriptを動作させ、常駐(デーモン化)するためのツール
- vite
 - nuxt.js(Vue)の開発者が作ったもので、当然相性がいいツール。JavaScriptをバンドル(まとめる)とともに、ブラウザではサポートしていない一部の構文を変換するなどを行う

```
npm install express
npm install --save-dev vite
```

turbopackのエントリーポイントを作成する

クライアント側のメインとなるエントリーポイントを作成する。

ファイル名: vite.config.js

```
import { defineConfig } from 'vite'
import { resolve } from 'path'

export default defineConfig({
  root: resolve(__dirname, 'src'),
  build: {
    outDir: resolve(__dirname, 'dist'),
    emptyOutDir: true,
    rollupOptions: {
      output: {
        entryFileNames: `assets/[name].js`,
        chunkFileNames: `assets/[name].js`,
        assetFileNames: `assets/[name].[ext]`,
      }
    }
  },
  server: {
    proxy: {
      '/api': 'http://localhost:3000'
    }
  }
})
```

package.jsonのscript項目を書き換え、プログラムを実行できるように調整する。

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "start": "node server.js"
},
```

また、サンプルのファイルとして以下を用意する。

- src/index.html

```
<!DOCTYPE html>
<html lang="ja">
  <head>
    <meta charset="UTF-8" />
    <title>Vite + Express Sample</title>
    <link rel="stylesheet" href="/style.css" />
  </head>
  <body>
    <h1>Hello Vite + Express</h1>
    <script type="module" src="/main.js"></script>
  </body>
</html>
```

- src/style.css

```
body {
  font-family: sans-serif;
  background-color: #f0f0f0;
  padding: 2rem;
}
```

- src/main.js

```
console.log('hello');
console.log('Vite + Express is working!');

const api = await fetch('/api');
const data = await api.text();
console.log(data);
```

- server.js

これだけexpressのファイル。

```
import express from 'express'
import path from 'path'
import { fileURLToPath } from 'url'

const app = express()
const port = 3000

const __dirname = path.dirname(fileURLToPath(import.meta.url))
const distPath = path.join(__dirname, 'backend')
```

```
app.use(express.static(distPath))

app.get('/api', (req, res) => {
  res.send('Hello Express!!');
})

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`)
})
```

それぞれ実行してみる。

実行環境としては、「dev」と「start」の両方を立ち上げる。

```
npm run build
npm run dev
npm run start
```